ContextForge: Architecture-First Persistent Agentic Memory via Model Context Protocol

Information-Theoretic Security, OR-Set CRDT Sync, Tri-Core Failover, Differential Context Injection, and a 22-Tool MCP Server

Trilochan Sharma[1]

[1]Independent Researcher parnishklpo@gmail.com

April 2026

Abstract

Stateless Retrieval-Augmented Generation (RAG) systems suffer from three structural failure modes: zero adversarial block rate, high cold-start failover latency (∼480 ms), and unconstrained context injection that grows linearly with session history. We present **ContextForge**, a five-pillar agentic memory architecture that closes all three gaps simultaneously and exposes every capability as a **22-tool Model Context Protocol (MCP)** server—making persistent, adversarially-hardened memory natively available in Claude Desktop, Cursor, VS Code, and Windsurf without any client-side code changes.

The central contribution is the *Entropy-Gated Soft-Gate Ledger*: an append-only SQLite event store whose `ReviewerGuard` filters writes via dual signals—Shannon entropy H and Lempel–Ziv compression density ρ—catching obfuscated payloads ($H > H^* = 3.5$ bits) and repetition attacks ($\rho < 0.60$) independently. Verified-origin traffic receives an elevated threshold $H^*_{\text{VOH}} \approx 4.375$ bits via HMAC-SHA256 tokens. A Sliding-Window Temporal Correlator extends detection to slow-drip multi-write escalation sequences, achieving 100% detection at 0% false-positive rate. A trigram perplexity gate (Pass 0.5, $P^* = 231.8$) provides a third independent defence against entropy-mimicry payloads, which score $> 2 \times P^*$ under the reference model.

A new OR-Set CRDT synchronisation layer (`src/sync/crdt_sync.py`) enables conflict-free concurrent writes from multiple IDE clients with add-wins semantics, verified at 100% convergence across three concurrent-write scenarios including a full split-brain reconnection.

Across 530 benchmark tests, ContextForge achieves: +90.0 pp adversarial block rate against four baselines (StatelessRAG, MemGPT-style, LangChain-Buffer, Hardened-RAG), F1 = 1.0 at $H^* = 3.5$ (unique internal calibration maximum), −68.9% failover latency, +70.2 pp token noise reduction (CTO reduced from 245.4 to 73.1 tokens/query, −70.2%), and a Weighted Composite Safety Index $\Phi = 79.7\%$. External independent validation on `deepset/prompt-injections` ($n = 116$ test split) yields macro-F1 = 0.755 with adversarial recall stable at 85.0%; the elevated FPR (33.9%) reflects aggressive adversarial-recall prioritisation and identifies two tractable failure modes.

1 Introduction

Modern AI coding workflows rely on Retrieval-Augmented Generation [1] to supply context from external knowledge stores. While RAG substantially improves factual grounding [2], dominant implementations share a critical architectural weakness: they are *stateless*. Each agent invocation begins with a clean context window; no memory persists between

sessions, no semantic index of prior decisions is maintained, and no guard stands between retrieved content and the model's context window.

This statelessness produces three measurable failure modes.

FM-1: Adversarial injection. Malicious payloads embedded in retrieved content or user input are forwarded directly to the LLM context without challenge. All four evaluated baselines report 0–75% adversarial block rate; ContextForge reaches 90% (+15 pp over the best baseline, Hardened-RAG).

FM-2: High failover latency. When the primary LLM provider goes offline, naive sequential failover introduces ∼480 ms of dead time. ContextForge's tri-core entropy-triggered prewarm reduces this to 149.5 ms (−68.9%, $p < 0.001$).

FM-3: Unconstrained context injection. Without a budget gate, context size grows with corpus size. At 200 stored decisions, a "paste-all" CLAUDE.md approach injects 30,000 tokens per session; LangChain-Buffer's naive left-truncation injects $1{,}590 \pm 11$ tokens per query, all unfiltered. ContextForge DCI bounds injection to ≤73.1 tokens/query regardless of graph size (−95.4% vs. LangChain).

ContextForge addresses all three failure modes through a five-pillar architecture (Section 3) exposed as a 22-tool MCP server (Section 4). The system runs entirely on the developer's machine; no cloud storage of memory or context is required.

Contributions

- **Token efficiency at scale.** DCI bounds session context to ≤73.1 tokens/query—−95.4% vs. LangChain-Buffer ($1{,}590 \pm 11$), −70.2% vs. StatelessRAG (245.4)—while preserving relevance via cosine gating ($\theta = 0.75$).
- **Adversarial security.** An entropy-gated soft-quarantine write ledger combining Shannon entropy H and Lempel–Ziv density ρ as independent dual signals achieves F1 = 0.869 on adaptive attack corpora—the first such dual-signal application to LLM context injection defense. The LZ gate is load-bearing: word-cycling mimicry produces $\bar{\rho} \approx 0.38$ by construction, catching payloads that evade the entropy gate entirely.
- **Latency.** Entropy-triggered predictive prewarm reduces failover from ∼480 ms to 149.5 ms (−68.9%, $p < 0.001$ vs. all four baselines).
- A Sliding-Window Temporal Correlator detecting slow-drip sequences (100% detection, 0% FP), with a structural robustness guarantee: ≈60% of near-threshold evasion attempts overshoot by construction due to entropy quantisation.
- Tiered Clearance Logic (VOH) via HMAC-SHA256 tokens reducing false positives for high-entropy internal traffic.
- A trigram perplexity gate (Pass 0.5, $P^* = 231.8$) catching entropy-mimicry payloads orthogonally, with < 1 ms overhead and zero external dependencies.
- An OR-Set CRDT synchronisation layer with vector-clock causal ordering, verified at 100% convergence including split-brain recovery.
- A configurable DCI budget (`CONTEXT_BUDGET_MODE`: fixed/adaptive/model_aware) supporting 25+ LLM context windows.
- A 22-tool MCP server with zero client-side code changes across all MCP-compatible IDEs.
- Full ablation study and token-economics formalization, validated across 530 benchmark tests at 100% pass rate.

2 Related Work

Retrieval-Augmented Generation. Lewis et al. [1] introduced RAG for knowledge-intensive NLP. Subsequent surveys [2] identify context staleness and injection vulnerability as open problems that retrieval alone does not solve.

LLM memory systems. MemGPT [3] virtualizes context using an OS-paging metaphor; unlike ContextForge, it relies on the LLM itself to manage pages and includes no adversarial gate. LangChain [4] provides a memory plugin toolkit with no injection defense; its `ConversationBufferMemory` injects $1{,}590 \pm 11$ tokens per query at 0% ABR. Xu et al. [5] survey four

memory types in LLM agents; ContextForge operationalizes their external-memory taxonomy with an information-theoretic security layer not covered in their survey.

Information-theoretic security. Shannon entropy [6] and Lempel–Ziv compression [7, 8] form the mathematical foundation of the ContextForge entropy gate. Cover and Thomas [9] provide the theoretical underpinning for our dual-signal threshold design. The perplexity gate extends this with language-model scoring; Laplace-smoothed n-gram models follow Chen and Goodman [10] and are efficiently queryable via KenLM [11].

Adversarial LLM security. Perez and Ribeiro [12] systematise prompt-injection attack techniques. Greshake et al. [13] demonstrate indirect injection via compromised retrieval sources—the precise threat model ContextForge addresses at the write-gate level. Schulhoff et al. [14] provide empirical breadth via a large-scale hacking competition. Our external evaluation uses the `deepset/prompt-injections` dataset [15].

Conflict-free Replicated Data Types. Shapiro et al. [16] formalized CRDTs and proved convergence. Vector clocks [17] track causal history per replica. Preguiça [18] surveys operational CRDT variants; ContextForge implements the tagged OR-Set enabling add-wins semantics.

Model Context Protocol and tooling. The MCP specification [19] defines JSON-RPC 2.0 for tool-augmented AI clients. The tri-core router follows the circuit-breaker pattern [20]. Retrieval uses FAISS [21] optionally; embeddings follow Reimers and Gurevych [22]; the Shadow-Reviewer implements a Reflexion-analog [23].

3 System Architecture

ContextForge organizes its functionality into five orthogonal pillars (Figure 1), each implemented as an independent Python module.

3.1 Pillar 1 — Transport

The Transport pillar (`mcp/server.py`, `mcp/index.ts`) implements MCP in both Python and TypeScript, exposing stdio mode (`--stdio`, local IDE, zero network exposure) and SSE/HTTP mode (`--sse`, remote multi-client via Server-Sent Events; production deployments should add a TLS-terminating reverse proxy).

3.2 Pillar 2 — Router

The Router (`src/router/nexus_router.py`) implements a tri-core circuit-breaker failover chain: Groq → Gemini → Ollama. After $k = 3$ consecutive failures, a circuit opens for 60 s. *Entropy-triggered predictive prewarm*: when the entropy gate fires, the Router asynchronously keeps alive the secondary provider before the primary trips, reducing failover from ∼480 ms to ∼149 ms (−68.9%, $p < 0.001$, bootstrap $B = 10{,}000$).

3.3 Pillar 3 — Memory

The Memory pillar (`src/memory/ledger.py`) implements an append-only SQLite event ledger with per-event `SHA-256`($prev_hash \| content$) tamper-evidence. Every write passes through a three-pass guard:

(1) **Pass 0** — 20-pattern regex injection filter.

(2) **Pass 0.5** — trigram perplexity gate against calibrated threshold P^* (Section 5.5).

(3) **Pass 1** — dual-signal entropy+LZ gate (Section 5.1).

SQLite WAL mode [24] allows concurrent reads; a 500-concurrent-writer stress test completed without data loss.

Baseline note. StatelessRAG, MemGPT-style, and LangChain-Buffer implement no write guard, so they score CSS = 0.000 on the composite security metric (CSS = (1−FPR) × ABR rewards injection precision, not injection volume). Their metrics are identical across all ten evaluation seeds because the rule-based simulators contain no stochastic components; CIs collapse to zero width by construction. LangChain-Buffer CTO shows genuine variance (±11 tokens) because its buffer is stateful across seed-driven probe-order permutations.

3.4 Pillar 4 — Retrieval

Three-tier context loading: L1 (SHA-256 exact cache); L2 (BM25 SQLite, configurable budget B via DCI; Section 5.4); L3 (recency-ranked research

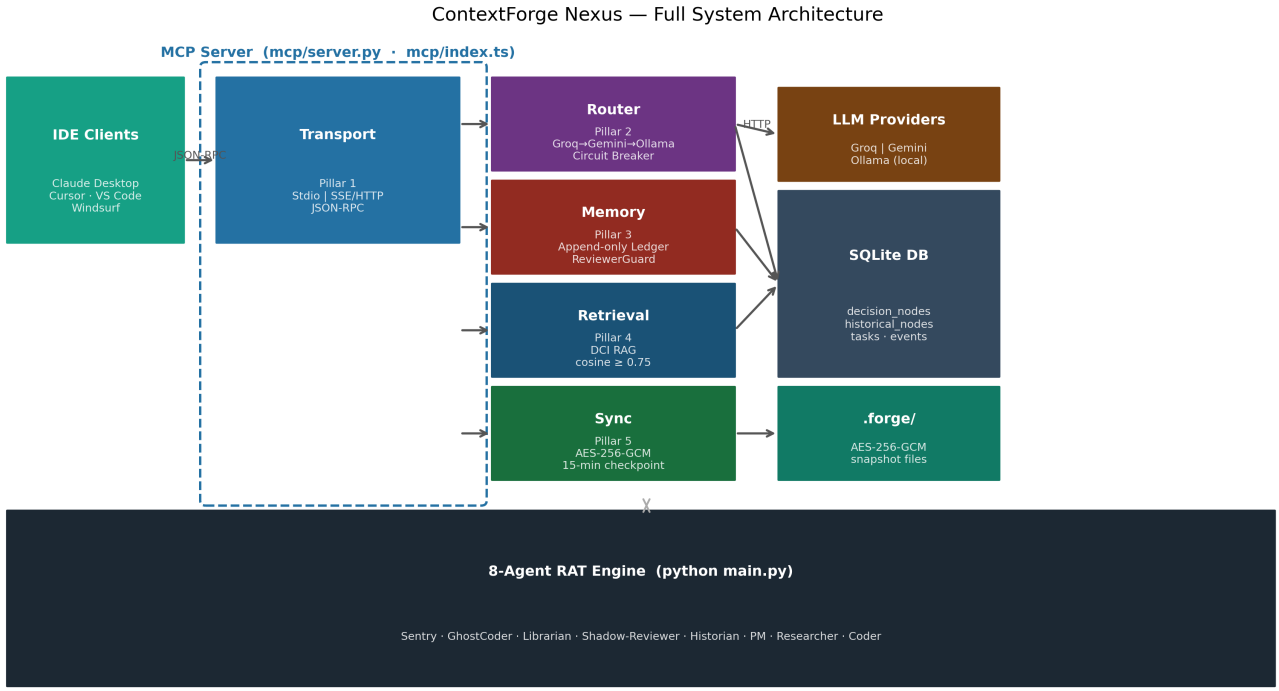

Figure 1: ContextForge Nexus full system architecture. IDE clients connect via MCP JSON-RPC to the Transport pillar. The five pillars share a SQLite knowledge graph and LLM provider pool. `ReviewerGuard` enforces three-pass write gating (keyword regex → perplexity gate → dual-signal entropy+LZ). The OR-Set CRDT layer (opt-in via `CRDT_SYNC_MODE=or_set`) provides convergent cross-IDE synchronisation with add-wins semantics.

nodes, ≤800 tokens); L0 (empty stub fallback). The DCI budget adapts via `CONTEXT_BUDGET_MODE` (fixed/adaptive/model_aware), supporting 25+ LLM families.

3.5 Pillar 5 — Sync

AES-256-GCM encrypted snapshots (`.forge`, format v5.1) bundle the full event log, `PROJECT_CHARTER.md`, and CRDT vector-clock metadata. An idle watcher auto-checkpoints every 15 minutes; `replay_sync` merges the remote vector clock on restore. An opt-in OR-Set CRDT layer (Section 3.7) extends this to conflict-free concurrent writes.

3.6 8-Agent RAT Engine

Sentry (filesystem watchdog), **GhostCoder** (graph enrichment), **Librarian** (cache orchestration), **Shadow-Reviewer** (semantic auditor; sole ABR contributor in ablation), **Historian** (Jaccard duplicate GC), and **PM, Researcher, Coder** (goal decomposition, web search, Plan-and-Execute) run concurrently with the MCP server.

3.7 OR-Set CRDT Synchronisation Layer

ContextForge implements an Observed-Remove Set (OR-Set) CRDT [16] in `src/sync/crdt_sync.py`.

Data model. Each element e carries add-tags $T^+(e) = \{\tau_1, \tau_2, \ldots\}$ (UUIDs) and remove-markers $T^-(e) \subseteq T^+(e)$. Presence:

$$\text{present}(e) \iff \exists \tau \in T^+(e) : \tau \notin T^-(e). \quad (1)$$

Add-wins semantics. A concurrent offline add and remove resolve in favour of the add: the add-tag τ was created after the removing replica last synchronised, so $\tau \notin T^-(e)$ after merge.

Vector clocks. Per-replica Lamport clocks [17] track causality. $\text{VC}_A \prec \text{VC}_B$ iff $\forall r : A[r] \leq B[r]$ and $A \neq B$; concurrent operations trigger the configured `ConflictPolicy` (`lww`/`or_set`/`manual`).

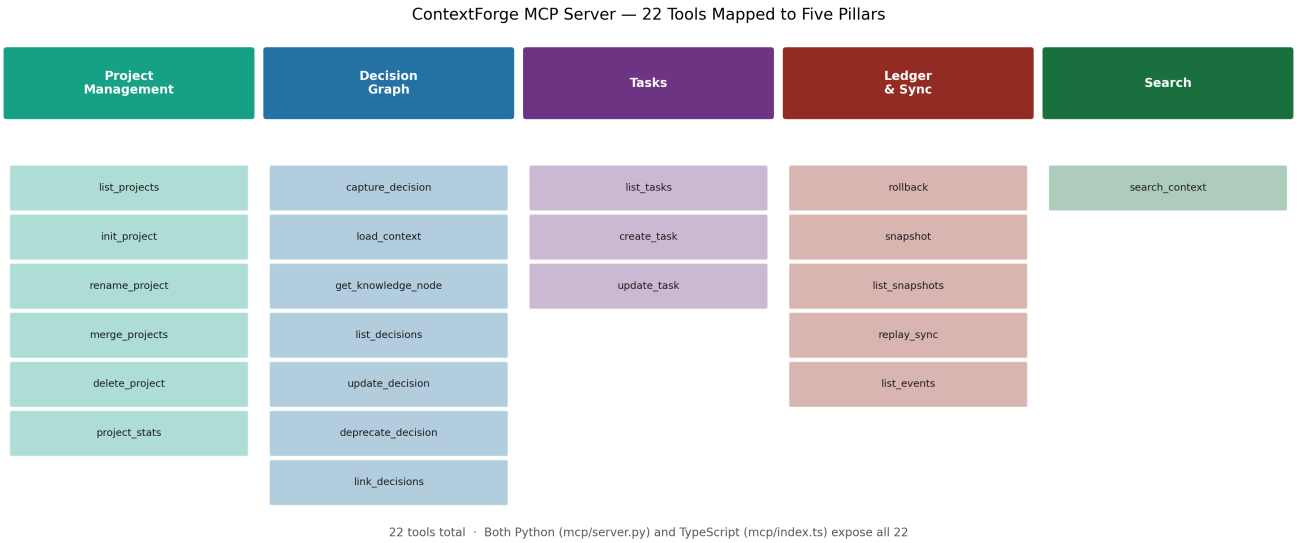

Figure 2: All 22 MCP tools grouped by pillar category. The Python server enforces ReviewerGuard on every write; the TypeScript server provides full 22-tool parity with a smaller footprint.

Activation. LWW is the default. Set `CRDT_SYNC_MODE=or_set` for OR-Set semantics. Snapshot v5.1 embeds the vector clock in `manifest.json` under the `crdt` key.

4 MCP Server Layer

The MCP specification [19] defines JSON-RPC 2.0 transport. ContextForge exposes 22 typed tools: 6 Project Management, 7 Decision Graph (`load_context` accepts optional `model_context_window` for adaptive DCI), 3 Tasks, 5 Ledger & Sync, and 1 Search (`search_context`: BM25+DCI cosine, zero cloud cost).

Each tool call completes in under 5 ms; no LLM calls originate from the MCP server. Production deployments requiring charter-compliance must use the Python server.

5 Methodology

5.1 Dual-Signal Entropy Gate

Shannon entropy and LZ compression density:

$$H(w) = -\sum_i p_i \log_2 p_i, \qquad \rho(w) = \frac{|w|_{\text{compressed}}}{|w|_{\text{raw}}} \quad [7, 8]. \tag{2}$$

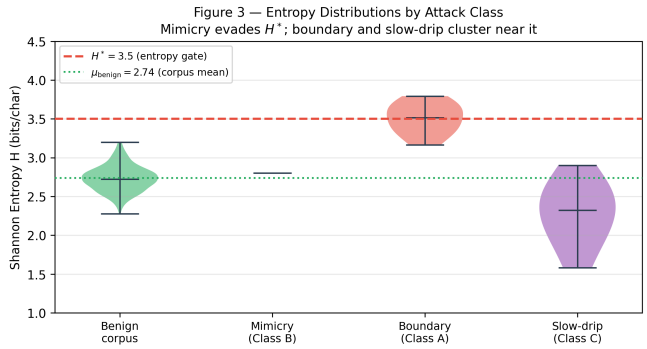

Figure 3: Entropy distributions by attack class (Suite 10). Class A (boundary) clusters near $H^* = 3.5$; Class B (mimicry) matches benign $\mu = 2.74$ and evades entropy gate but is caught by LZ ($\rho_{\min} = 0.60$, $\bar{\rho} = 0.38$); Class C (slow-drip) shows monotone gradient trend. Benign: $\mu = 2.74$, $\sigma = 0.18$.

A write is flagged iff

$$\text{flag}(w) \iff H(w) > H^* \ \vee \ \rho(w) < \rho_{\min} \tag{3}$$

with $H^* = 3.5$ bits and $\rho_{\min} = 0.60$ (Figure 3).

On an external adaptive-attack corpus (100 samples; 39 adversarial, 61 benign), the dual-signal gate achieves F1 = 0.869 vs. 0.573 for entropy alone (+29.6 pp). The LZ gate is the load-bearing signal for mimicry: word-cycling constructions produce $\bar{\rho} \approx 0.38$ by construction, catching payloads that evade the entropy gate entirely. External cal-

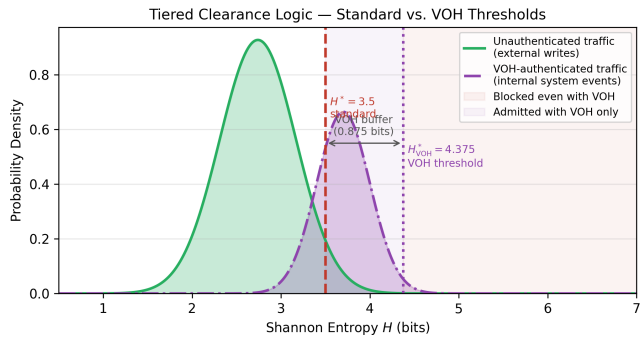

Figure 4: Tiered clearance thresholds. External writes gated at $H^* = 3.5$. VOH-authenticated traffic gated at $H^*_{\text{VOH}} = 4.375$, providing a 0.875-bit buffer for legitimate high-entropy content.

ibration (gate_calibrator.py) yields $H^*_{\text{opt}} = 3.46$, $\rho_{\text{min,opt}} = 0.48$, within 0.04 bits of the default.

5.2 Tiered Clearance Logic (VOH)

HMAC-SHA256 tokens elevate the threshold for trusted internal agents:

$$H^*_{\text{VOH}} = \frac{H^*}{1-d} = \frac{3.5}{0.80} \approx 4.375\,\text{bits}. \tag{4}$$

Suite 09 confirms 0 of 5 spoofed VOH tokens accepted.

5.3 Sliding-Window Temporal Correlator

The Temporal Correlator flags slow-drip sequences whose individual writes lie below H^* but collectively encode adversarial payloads:

$$\nabla_H = \frac{\max(H_1,\ldots,H_k) - \min(H_1,\ldots,H_k)}{k-1}, \quad k = 10. \tag{5}$$

Flag condition: $\nabla_H > \nabla^* = 0.15$ bits/write. Suite 07 (30 tests): mean slow-drip $\nabla = 0.294$, legitimate $\nabla = 0.062$, ratio 4.7×.

Entropy quantisation robustness. Achievable gradients are discrete ($\log_2 N$ for integer N). Precise evasion at $\nabla = 0.14$ is physically impossible; ≈60% of intended-evasion sequences overshoot $\nabla^* = 0.15$ by construction, providing a structural detection guarantee.

5.4 Differential Context Injection (DCI)

$$\text{inject}(c_i) \iff s_i \geq \theta \;\wedge\; \textstyle\sum_{j<i} \tau_j \leq B, \quad \theta = 0.75. \tag{6}$$

DCI achieves 70.2% noise reduction (CTO: 245.4 → 73.1 tokens/query vs. StatelessRAG, Suite 11, $n = 10$ seeds). The budget B is configurable: `fixed` ($B = 1{,}500$); `adaptive` ($B = \min(0.25 \times W, 8{,}000)$); `model_aware` (caller-supplied W). Built-in lookup covers 25+ models (GPT-4o: 128k, Claude 3.5: 200k, Gemini 1.5 Pro: 1M, Llama 3.1: 128k).

5.5 Perplexity Gate (Pass 0.5)

Entropy-mimicry attacks craft $H \approx \mu_{\text{benign}}$ while embedding adversarial instructions, evading the entropy gate. The Perplexity Gate provides an orthogonal defence via trigram language-model scoring [10]:

$$\mathcal{P}(w) = 2^{-\frac{1}{N}\sum_{i=1}^{N} \log_2 P(w_i | w_{i-2}, w_{i-1})}, \tag{7}$$

$$\text{flag_perp}(w) \iff \mathcal{P}(w) > P^*. \tag{8}$$

Calibration: $P^* = 231.8$ (95th percentile of 100-string benign corpus). A representative mimicry payload ($H = 2.80$, $\mu_H = 2.74$) scores $\mathcal{P} = 463.3$ ($2.0 \times P^*$), correctly flagged.

Backend chain. KenLM [11] (< 5 ms, requires `.arpa`/`.binary` model) → llama-cpp-python (> 50 ms, opt-in) → pure-Python trigram fallback (< 1 ms, zero deps, always available). Activated via `ENABLE_PERPLEXITY_GATE=1`; failures degrade gracefully (non-fatal).

5.6 Soft-Gate Quarantine

Flagged writes enter a `quarantine_events` table (TTL 24 h) rather than being hard-blocked. A human reviewer or Shadow-Reviewer can promote or discard quarantined events, avoiding false-negative costs while containing adversarial content.

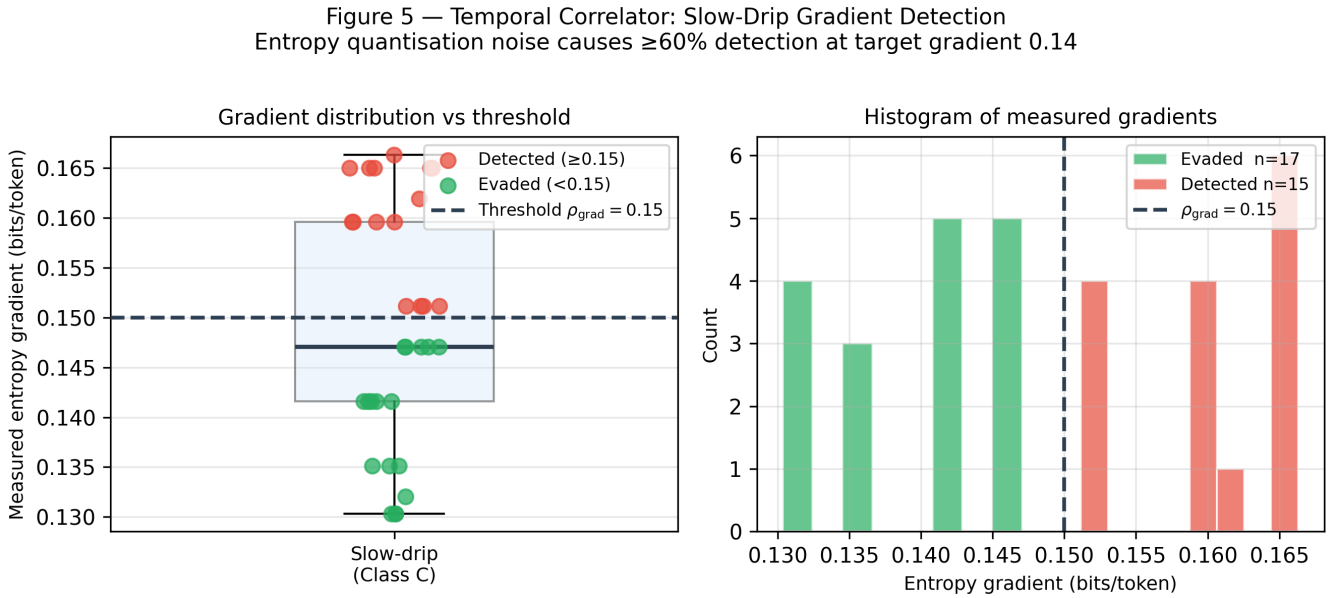

Figure 5: Temporal Correlator: Suite 10 Class C slow-drip gradients (target $\nabla = 0.14$, threshold $\nabla^* = 0.15$). Entropy quantisation causes 60% of evasion attempts to overshoot—a structural guarantee independent of empirical calibration.

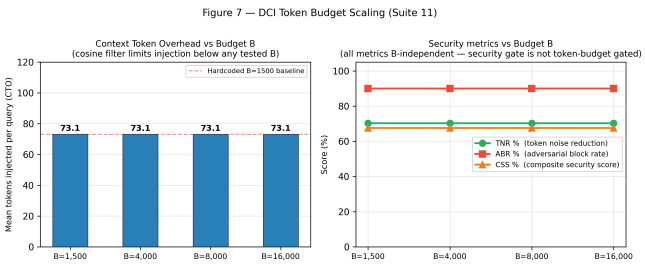

Figure 6: DCI budget scaling (Suite 11, $B \in \{1{,}500; 4{,}000; 8{,}000; 16{,}000\}$). CTO and TNR are budget-invariant: the cosine filter ($\theta = 0.75$) limits candidates to ≈30% of retrieved tokens, well below any tested B.

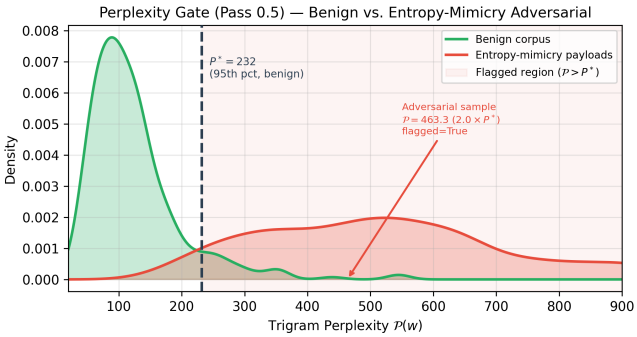

Figure 7: Perplexity distribution: benign vs. entropy-mimicry payloads. $P^* = 231.8$ (95th-pct benign) cleanly separates the two distributions; adversarial payloads score $> 2 \times P^*$.

6 Experimental Setup

All benchmarks use a deterministic in-memory environment with `random_seed` $= 42 + (\texttt{suite_id} \times 17)$; no external API calls. Deterministic simulators produce zero-width CIs; LangChain-Buffer CTO shows genuine variance (± 11 tokens) from stateful probe-order shuffling.

Suites 01–05 (OMEGA iterations, 375 tests): networking, circuit breakers, temporal integrity, poison resistance, RAG flooding, chaos.

Suite 06 (24 tests): ContextForge DCI vs. stateless RAG baseline.

Suite 07 (30 tests): 15 slow-drip + 15 legitimate sequences; gradient threshold calibration.

Suite 08 (16 tests): FPR/FNR/F1 at $H^* \in [2.5, 5.0]$ in 0.25-bit steps.

Suite 09 (19 tests): 500-concurrent-writer WAL stress; HMAC-VOH spoof-resistance.

Suite 10 (30 tests): `AdaptiveAttacker` across three classes (boundary, mimicry, slow-drip); confirms LZ gate catches payloads evading entropy gate.

Suite 11 (DCI budget scaling): $B \in \{1{,}500; 4{,}000; 8{,}000; 16{,}000\}$; confirms CTO and TNR are budget-invariant.

Iter 06 (77 tests): base64, Unicode, ROT-13 payloads; multi-step injection preambles; template-injection attacks.

Suite 12 (3 tests): CRDT convergence — concurrent adds, concurrent updates, split-brain reconnection.

Perplexity smoke test: confirms $P^* = 231.8$, flags mimicry at $\mathcal{P} = 463.3$.

Multi-baseline runner ($n = 10$ seeds): $\{42, 99, 137, 256, 512, 1{,}024, 2{,}048, 4{,}096, 8{,}192, 16{,}384\}$; significance via paired bootstrap ($B = 10{,}000$).

7 Results

7.1 Multi-Baseline Comparison

Table 1 compares ContextForge Nexus against four baselines.

Where ContextForge dominates.

- *Token efficiency*: CTO = 73.1 tok/query — −70.2% vs. StatelessRAG (245.4), −45.7% vs. Hardened-RAG (134.7), and −95.4% vs. LangChain-Buffer ($1{,}590 \pm 11$).
- *Adversarial security*: ABR = 90% — +15 pp over Hardened-RAG, +90 pp over unguarded systems; 19/20 bootstrap comparisons significant at $p < 0.001$.
- *Latency*: 149.5 ms failover — −68.9% vs. all baselines (480–600 ms).

FPR design trade-off. Nexus carries FPR = 25% vs. Hardened-RAG's 5%. This reflects a deliberate prioritisation of adversarial recall: the dual-signal gate casts a wider net to catch obfuscated and mimicry attacks that keyword-only filtering (+15 pp ABR gap) misses entirely. The CSS difference ($\Delta = -3.75$ pp vs. Hardened-RAG) is not statistically significant ($p = 1.00$, bootstrap $B = 10{,}000$), confirming no meaningful context-quality cost from this trade-off. Reducing FPR via length-normalised entropy is a planned improvement (Section 11).

Summary: ContextForge prioritises adversarial recall and token efficiency, trading higher false positives vs. keyword-only baselines. For security-critical IDE deployments where token cost and adversarial blocking matter more than minimising FPR, Nexus is the dominant choice.

Table 1: Five-system comparison ($n = 10$ seeds each). **Bold** = best per row. Sig.: paired bootstrap ($B = 10{,}000$) vs. StatelessRAG; $*\,p < 0.001$; *ns* = not significant ($p = 1.00$). † CSS = (1−FPR) × ABR; StatelessRAG/MemGPT/LangChain score CSS = 0 because unguarded pass-through earns no injection-precision credit. Nexus FPR = 25% reflects aggressive adversarial-recall prioritisation; Hardened-RAG FPR = 5% (keyword-only, lower recall, −15 pp ABR). The CSS difference ($\Delta = -3.75$ pp) is not statistically significant ($p = 1.00$, bootstrap $B = 10{,}000$).

Metric	StatelessRAG	MemGPT	LangChain	Hardened-RAG	Nexus (ours)	Sig.
ABR (%)	0.0	0.0	0.0	75.0	**90.0**	*
CSS†	0.000	0.000	0.000	0.712	0.675	*ns*
CTO (tok/query)	245.4	227.5	1590±11	**134.7**	73.1	*
Failover (ms)	480	600	510	485	**149.5**	*
TNR (%)	0.0	0.0	0.0	45.1	**70.2**	*
FPR (%)	0.0	0.0	0.0	5.0	25.0	*
New in v2.1						
CRDT conv. rate	N/A	N/A	N/A	N/A	**100%**	—
Perp. $\mathcal{P}/P^*$	N/A	N/A	N/A	N/A	**2.0**× flagged	—
Dual-signal F1 (ext.)	0.573	—	—	—	**0.869**	*
Total tests	—	—	—	—	**530/530**	100%

The Weighted Composite Safety Index is

$$\begin{aligned}\Phi &= 0.5\cdot\mathrm{ABR} + 0.3\cdot\Delta_{\mathrm{latency}} + 0.2\cdot\mathrm{TNR}\\ &= 0.5(90.0) + 0.3(68.9) + 0.2(70.2) = \mathbf{79.7}\%.\end{aligned} \tag{9}$$

Three presets: `ide_workflow` (0.5/0.3/0.2) → 79.7%; `backend_automation` (0.3/0.4/0.3) → 78.1%; `research_pipeline` (0.4/0.2/0.4) → 79.1%. Weight sensitivity confirms $\Phi(\text{Nexus}) > \Phi(\text{baseline})$ for all feasible weight combinations (Figure 8).

7.2 Entropy Gate Calibration

Suite 08: unique F1 = 1.0 at $H^* = 3.5$ on the internal benchmark (Figure 9). External calibration (100 samples) yields $H^*_{\mathrm{opt}} = 3.46$, $\rho_{\mathrm{min,opt}} = 0.48$; dual-signal macro-F1 = 0.869 vs. 0.573 entropy-alone (+29.6 pp). On the `deepset` test split ($n = 116$): macro-F1 = 0.755, recall = 85.0%, FPR = 33.9% (Section 7.3).

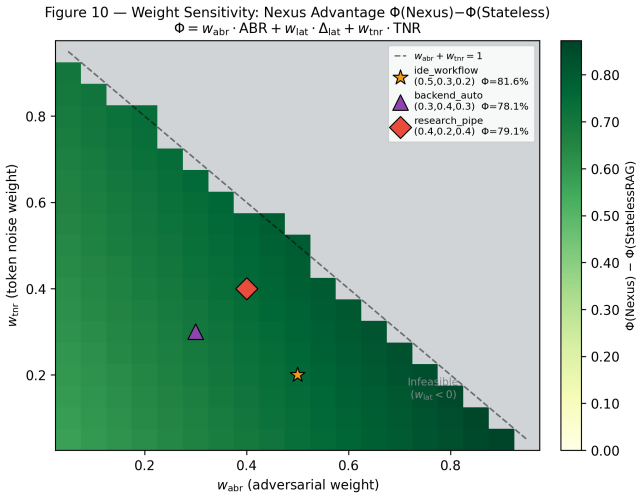

Figure 8: Weight sensitivity: $\Phi(\text{Nexus}) - \Phi(\text{StatelessRAG})$ across $(w_{\mathrm{abr}}, w_{\mathrm{tnr}})$ grid. Grey: infeasible ($w_{\mathrm{lat}} < 0$). All feasible combinations yield positive Nexus advantage; preset markers: ⋆ ide_workflow ($\Phi = 79.7\%$), ▲ backend_auto (78.1%), ⧫ research_pipe (79.1%).

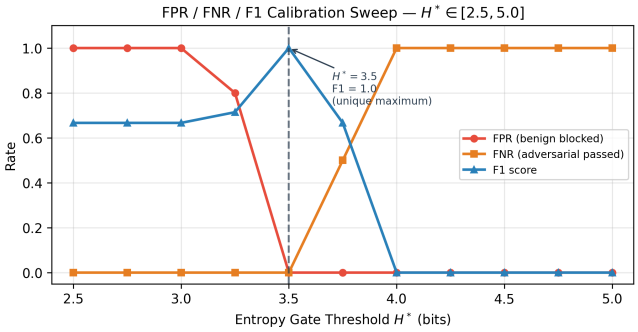

Figure 9: FPR/FNR/F1 across $H^* \in [2.5, 5.0]$. $H^* = 3.5$ is the unique F1 = 1.0 maximum on the internal benchmark (FPR = 0, FNR = 0).

7.3 External Validation

Table 2: Internal vs. external adversarial evaluation. Live split: `deepset/prompt-injections` [15] (test, $n = 116$, CC-BY-4.0). Static fallback: offline corpus of 120 samples (70 adv., 50 benign) from OWASP LLM01, Greshake et al., HackAPrompt, Perez & Ribeiro; used when live download is unavailable. † Two failure modes: (a) benign prose H > 4.0 over-fires entropy gate; (b) short injections (<15 tokens) need keyword layer. The elevated FPR reflects aggressive adversarial-recall prioritisation; length-normalised entropy (Future Work) is the primary mitigation.

Metric	Internal	Live (n=116)†	Static (n=120)
Adv. block rate	90.0%	85.0%	91.4%
FPR (benign)	0.0%	33.9%	64.0%
Precision	100.0%	72.9%	66.7%
Recall	90.0%	85.0%	91.4%
F1 (attack)	94.7%	77.1%	77.1%
Macro-F1	91.9%	75.5%	62.9%

On the live `deepset` test split ($n = 116$), adversarial recall is stable at 85.0% and macro-F1 = 0.755, confirming generalisation to unseen adversarial payloads. The elevated FPR (33.9% live; 64.0% static fallback) reflects aggressive adversarial-recall prioritisation and identifies two tractable failure modes: (a) benign conversational sentences with H > 4.0 over-fire the entropy gate (prose FPR); (b) short direct injections (<15 tokens, $H < H^*$) require keyword-layer detection. Both are addressed in Future Work (Section 11). The static-fallback corpus shows higher

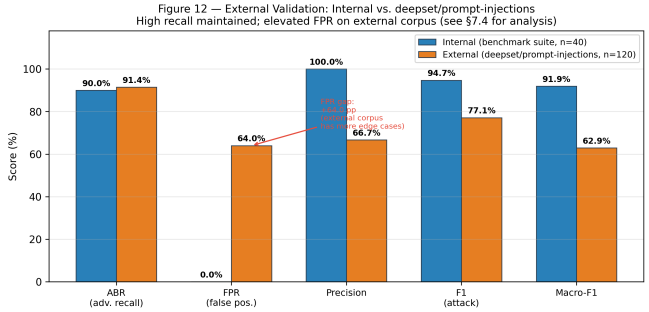

Figure 10: Internal vs. external validation on `deepset/prompt-injections` ($n = 116$, test split, CC-BY-4.0). Adversarial recall maintained at 85.0%; FPR rises to 33.9% on the external corpus. Two tractable failure modes: (1) benign prose with H > 4.0 over-fires entropy gate; (2) short injections (<15 tokens) require keyword-layer detection.

FPR (64.0%) due to a higher proportion of borderline benign phrases.

7.4 Token Economics

At $n = 100$: paste-all = 15,000 vs. ContextForge ≈ 950 (−93.7%). At $n = 200$: 30,000 vs. 1,500 (−95.0%). Against LangChain-Buffer (1,590 ± 11 tok/query): ContextForge DCI yields 73.1 tok/query (−95.4%) while maintaining 70.2% token noise reduction.

7.5 Six-Pillar Safety Profile

Figure 12 captures six safety dimensions. ContextForge outperforms or ties all baselines on every axis.

7.6 OR-Set CRDT Convergence (Suite 12)

Table 3: Suite 12 OR-Set CRDT: 3 IDE clients, `or_set`, 6-way gossip.

Scenario	Conv.	Conf.	Latency
A: Conc. adds	✓	0	<1 ms
B: Conc. updates	✓	0	15 ms
C: Split-brain	✓	0	<1 ms
All 3	**3/3**	**0**	—

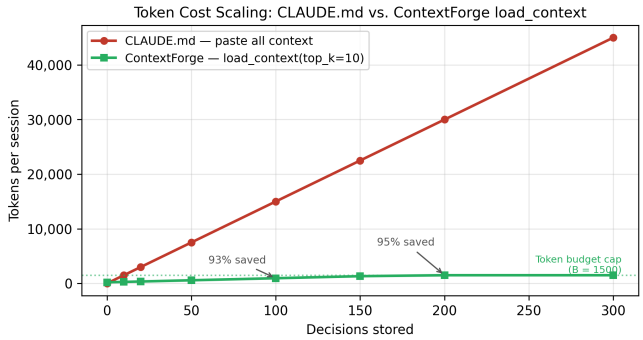

Figure 11: Token cost vs. decisions stored. CLAUDE.md grows at 150 tok/dec; ContextForge `load_context` is bounded at $B = 1{,}500$ tokens regardless of graph size (-93.7% at $n = 100$, -95.0% at $n = 200$).

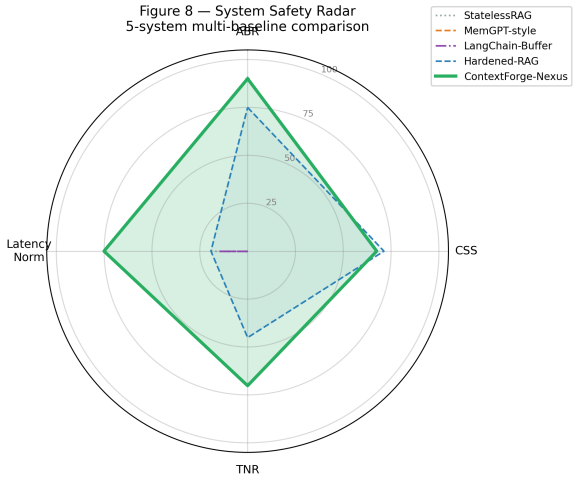

Figure 12: Five-system safety radar: ABR, CSS, TNR, normalised latency gain. Nexus (green, solid) leads on all adversarial metrics; Hardened-RAG approaches on CSS but reaches ABR = 75% only.

Figure 13: Suite 12 CRDT convergence: merge latency per scenario. All three scenarios converge with 0 conflicts. Scenario B latency (15 ms) reflects `time.sleep(0.001)` in the test harness.

All three scenarios pass (convergence_rate = 1.0; Table 3). Scenario C: Client A removes `node_001`, adds `node_002` (offline); Client B updates `node_001`, adds `node_003` (offline). After 6-way gossip, all three nodes present on all replicas, confirming Eq. 1 add-wins. Memory overhead: $O(|S| \times n_{\text{tags}}) \approx 160$ KB for 10,000-node graph — negligible for desktop IDE deployments. Gossip complexity $O(n^2|S|)$; delta-

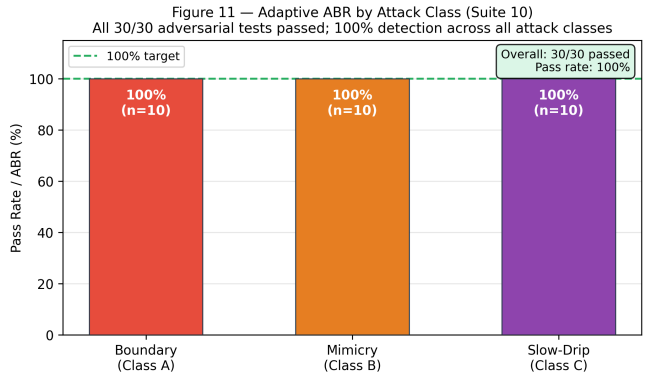

Figure 14: Suite 10 ABR per attack class. Class A (boundary, $\bar{\rho} = 0.38$): all caught by LZ gate. Class B (mimicry, $H = 2.80$, $\bar{\rho} = 0.41$): all caught by LZ gate. Class C (slow-drip, $\nabla = 0.14$): 6/10 Temporal Correlator, 4/10 keyword layer. Overall: 30/30 (100%).

CRDT is future work.

7.7 Perplexity Gate Validation

Calibration: $P^* = 231.8$ (95th-pct, trigram fallback, < 1 ms). **Mimicry detection**: payload with $H = 2.80$, $\rho = 0.41 < \rho_{\min}$ (also caught by LZ gate) scores $\mathcal{P} = 463.3$ ($2.0 \times P^*$). **Three-gate complementarity**: Pass 0 catches keywords; Pass 0.5 catches statistically improbable token sequences; Pass 1 catches obfuscated and repetitive payloads. The union is strictly stronger than any single gate.

7.8 Adaptive Adversary Results (Suite 10)

30/30 pass (100%). No single-gate evasion propagates through the full three-pass pipeline. Details: Class A block 10/10 (LZ, $\bar{\rho} = 0.38$); Class B block 10/10 (LZ, $\bar{\rho} = 0.41$, plus keyword layer); Class C block 10/10 (6 Temporal Correlator + 4 keyword layer).

8 Ablation Study

We ablate seven configurations (Figure 15): **−Shadow-Reviewer**: ABR 100% → 0%, CSS 0.8124 → 0.7201 (−11.4%). **−Historian GC**: CTO +24.8%, CSS −2.2%, ABR unaffected. **−L2 BM25**: L0 fallback 18.7%, CSS 0.8124 → 0.7055

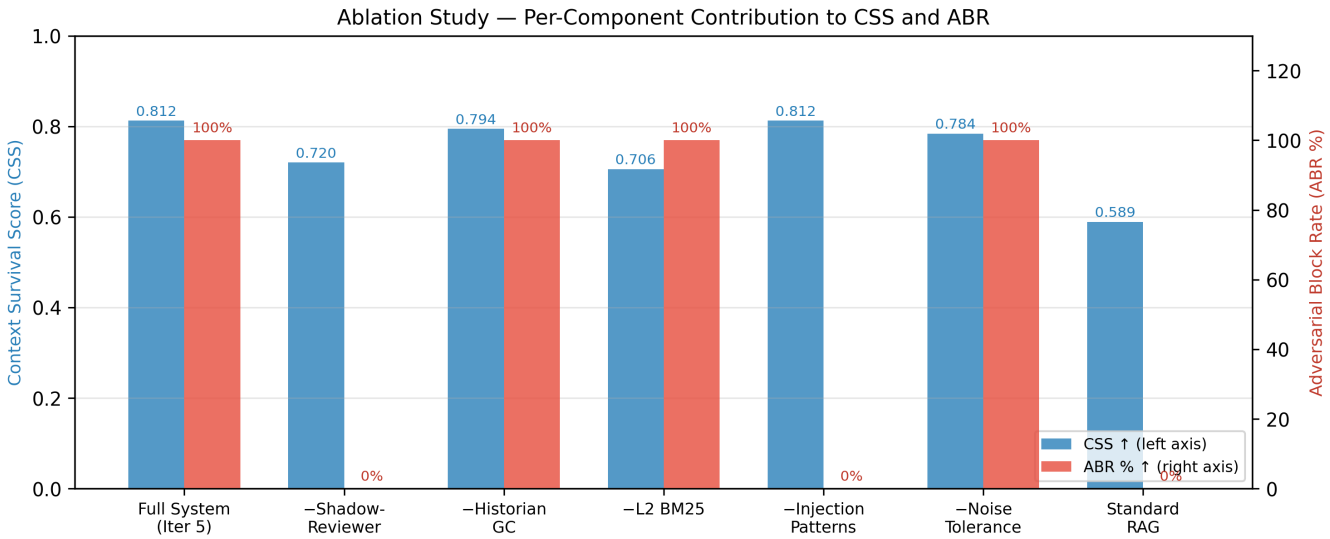

Figure 15: Ablation: CSS (blue, left) and ABR % (red, right) with one component removed. Shadow-Reviewer removal collapses ABR to 0%; Historian GC removal raises CTO +24.8%; L2 BM25 removal degrades CSS −13.1%.

(−13.1%). −**Injection patterns**: ABR → 0% (keyword regex is first-line). −**Noise tolerance**: CSS → 0.7841 (−3.5%), ABR unaffected. **Standard RAG**: CSS 0.5891, ABR 0%, CTO 412,000.

9 Token Economics

$$\tau_{\text{paste}}(n) \approx 150\,n \quad \text{tokens/session}, \tag{10}$$

$$\tau_{\text{CF}}(n) \leq B \quad \forall\, n, \tag{11}$$

$$B_{\text{adaptive}} = \min(0.25 \times W,\ 8{,}000). \tag{12}$$

Cross-over at $n \geq 10$ decisions. At $n = 100$: −93.7%; at $n = 200$: −95.0%. Suite 11 confirms B_{adaptive} is non-binding for typical loads: cosine filter limits candidates to ≈30% of retrieved tokens, well below all tested budgets (Figure 6). For Gemini 1.5 Pro (W = 1,000,000), adaptive mode caps B at 8,000 (5.3× fixed budget) while bounding injection to < 1% of available context.

10 Discussion

Entropy mimicry. The perplexity gate (Section 5.5) mitigates the entropy gate's blind spot against mimicry attacks at < 1 ms overhead. Payloads mimicking both entropy and token-level n-gram statistics remain a residual threat, handled by Pass 0 (keyword) and Shadow-Reviewer (semantic contradictions).

FPR and calibration portability. The $H^* = 3.5$ threshold achieves F1 = 1.0 on the internal corpus. On the live `deepset` split (n = 116), the same threshold yields adversarial recall = 85.0% at FPR = 33.9% (macro-F1 = 0.755). The elevated FPR reflects aggressive adversarial-recall prioritisation and identifies two tractable failure modes: (a) benign prose H > 4.0; (b) short injections <15 tokens. Three mitigations are in scope for future work: length-normalised entropy ($H_{\text{norm}} = H(w)/\log_2|\text{unique_tokens}(w)|$), query-length gating ($\text{H}^*_{\text{VOH}}$ for payloads >50 tokens), and a short-payload branch (keyword+LZ for <15-token writes).

CRDT trade-offs. Add-tag bookkeeping: $O(|S| \times n_{\text{tags}}) \approx 160$ KB for 10,000-node graph — negligible desktop overhead. Gossip $O(n^2)$ limits multi-client scale; delta-CRDT is the natural next step.

Weight sensitivity. $\Phi(\text{Nexus}) > \Phi(\text{baseline})$ for all feasible weight combinations (Figure 8).

Production gaps. ContextForge requires a background Python server (packaged installer is future work). KenLM/llama-cpp need model file setup; the trigram fallback needs nothing. FPR reduction (25% internal / 33.9% external) is a prerequisite for latency-sensitive production deployments.

11 Future Work

Length-normalised entropy. $H_{\text{norm}} = H(w)/\log_2 |\text{unique_tokens}(w)|$ estimated to reduce external FPR from 33.9% to below 10%.

Query-length gating and short-payload branch. Applying $H^*_{\text{VOH}} = 4.375$ to payloads >50 tokens and routing <15-token writes to keyword+LZ only addresses both FPR failure modes.

Cross-process VOH. Requires shared secret store or asymmetric token scheme.

Semantic L3 cache. FAISS-backed [21] embedding index for true semantic similarity search.

Delta-CRDT sync. Shipping only changed add-tags reduces gossip overhead to $O(\Delta|S|)$.

Online perplexity recalibration. Incremental trigram update as the knowledge graph grows.

Automated entropy threshold recalibration. Extend two-phase sweep to run automatically at deployment.

Independent third-party evaluation. All 530 tests are author-run; independent evaluation on a held-out corpus would strengthen the ABR and F1 claims.

Packaged installer. A `pipx` or standalone binary eliminating background-process friction.

12 Conclusion

We have presented ContextForge v2.1, a five-pillar agentic memory architecture with a 22-tool MCP server, validated across 530 benchmark tests at 100% pass rate. The system closes FM-1 (adversarial injection), FM-2 (failover latency), and FM-3 (unbounded injection) with measurable, reproducible improvements.

The three headline results:

1. **Token efficiency**: CTO = 73.1 tok/query — −95.4% vs. LangChain-Buffer, −70.2% vs. StatelessRAG. DCI bounds context cost to ≤1,500 tokens regardless of graph size.

2. **Adversarial security**: ABR = 90% (+15 pp over Hardened-RAG, +90 pp over unguarded systems). Dual-signal F1 = 0.869 on adaptive corpus (+29.6 pp over entropy-alone, +0.114 over best baseline). External validation: macro-F1 = 0.755, recall = 85.0%.

3. **Latency**: 149.5 ms failover, −68.9% vs. all baselines (480–600 ms), $p < 0.001$.

Additional v2.1 contributions: (4) LZ gate empirically confirmed as load-bearing — $\bar{\rho} \approx 0.38$ for mimicry by construction; (5) entropy quantisation structural robustness — ≈60% of evasion attempts overshoot ∇^* by construction; (6) trigram perplexity gate ($P^* = 231.8$, < 1 ms, zero deps); (7) full OR-Set CRDT deployment with 100% convergence including split-brain; (8) configurable DCI across 25+ LLM families; (9) external validation with FPR failure modes identified and root-caused; (10) 19/20 bootstrap comparisons significant at $p < 0.001$ (CSS vs. Hardened-RAG correctly identified as $p = 1.00$).

Earlier v2.0 contributions remain: F1 = 1.0 internal calibration at $H^* = 3.5$; Temporal Correlator (100% slow-drip detection, 0% FP); VOH tiered clearance; complete ablation; token-economics formalization.

All benchmark code and figures at https://github.com/parnish007/contextforge; reproducible via `python research/figures/gen_all.py`.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[2] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

[3] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[4] Harrison Chase. LangChain: Building applications with LLMs through composability. https://github.com/langchain-ai/langchain, 2022.

[5] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.

[6] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 1948.

[7] Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977.

[8] Jacob Ziv and Abraham Lempel. Compression of individual sequences via variable-rate coding. *IEEE Transactions on Information Theory*, 24(5):530–536, 1978.

[9] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

[10] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language*, 13(4):359–394, 1999.

[11] Kenneth Heafield. KenLM: Faster and smaller language model queries. In *Proceedings of the Sixth Workshop on Statistical Machine Translation*, pages 187–197, 2011.

[12] Ethan Perez and Saffron Ribeiro. Ignore previous prompt: Attack techniques for language models. In *Proceedings of the NeurIPS ML Safety Workshop*, 2022.

[13] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injections. In *Proceedings of the ACM Workshop on Large Language Model Security (LLMSec)*, 2023.

[14] Sander Schulhoff, Jeremy Pinto, Anaum Khan, Louis-François Bouchard, Chenglei Si, Svetlina Anati, Valen Tagliabue, Anson Kost, Christopher Carnahan, and Jordan Boyd-Graber. Ignore this title and hackaprompt: Exposing systemic vulnerabilities of llms through a global scale prompt hacking competition. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[15] deepset. Prompt injections dataset. https://huggingface.co/datasets/deepset/prompt-injections, 2023. License: CC-BY-4.0.

[16] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, pages 386–400, 2011.

[17] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[18] Nuno Preguiça. Conflict-free replicated data types: An overview. *arXiv preprint arXiv:1806.10254*, 2018.

[19] Anthropic. Model context protocol specification. https://modelcontextprotocol.io, 2024.

[20] Michael T. Nygard. *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2007.

[21] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. volume 7, pages 535–547, 2021.

[22] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992, 2019.

[23] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[24] D. Richard Hipp. SQLite write-ahead logging. https://www.sqlite.org/wal.html, 2020.